

Redes Neuronales II

Perceptrón, Red de Una Capa, Red Multicapa, Backpropagation,
Funciones de Activación, Clasificación Binaria y Multiclase
con TensorFlow/Keras y PyTorch

Estudiante: Joan Schneider Beltrán Delgado
Correo institucional: jsbeltran@ucundinamarca.edu.co
Asignatura: Deep Learning – Conceptos
Institución: Universidad de Cundinamarca – Posgrados
Fecha: Septiembre de 2026

Repositorio GitHub:
<https://github.com/JoanBeltranAlt/redes-neuronales-ii>

Contenido del documento

1. Introducción y alcance de la actividad
2. Notebook 1: Perceptrón, red de una capa, backpropagation, activaciones y clasificación binaria
 - Funciones de activación Sigmoide y ReLU
 - Dataset de clasificación binaria (semilunas, no linealmente separable)
 - Perceptrón simple rediseñado
 - Red neuronal de una capa rediseñada (Sigmoide vs. ReLU)
 - MLP desde cero con backpropagation (comparación Sigmoide vs. ReLU)
 - Implementación con TensorFlow + Keras
 - Implementación con PyTorch
 - Comparación de resultados entre todas las implementaciones
3. Notebook 2: Clasificación multiclase con MNIST
 - Carga y preprocesamiento del dataset MNIST
 - Red densa multiclase con TensorFlow + Keras
 - Red densa multiclase con PyTorch
 - Comparación de resultados y evidencias de ejecución
4. Resumen general de resultados
5. Conclusiones generales

1. Introducción y alcance de la actividad

Este documento reúne el trabajo hecho para la actividad "**Redes Neuronales II**", que continúa lo trabajado en la entrega anterior, "*Redes Neuronales Básicas*" (donde se construyeron desde cero un perceptrón, una red de una capa y una red multicapa).

Aquí se retoman esos tres modelos, se ajustan y se ponen a prueba en un contexto más exigente, y además se incorpora lo siguiente:

- El **perceptrón** y la **red de una capa**, puestos a prueba sobre un dataset que no se puede separar con una línea recta, para ver de primera mano su principal limitación.
- El algoritmo de **backpropagation**, comparando dos funciones de activación —**Sigmoide** y **ReLU**— y cómo cada una afecta la velocidad de aprendizaje de una **red neuronal multicapa**.
- Un problema de **clasificación binaria** sobre ese mismo dataset sintético (dos "semilunas" entrelazadas), resuelto primero a mano con NumPy y luego reproducido con dos frameworks especializados de Deep Learning.
- Un problema de **clasificación multiclase** sobre datos reales: el reconocimiento de dígitos escritos a mano del dataset **MNIST** (10 clases), implementado con redes densas (sin capas convolucionales) tanto en **TensorFlow + Keras** como en **PyTorch**.

Todo el desarrollo se hizo en dos notebooks de **Google Colab**, cuyo código, explicaciones, resultados y evidencias de ejecución se transcriben completos en las siguientes secciones. Ambos notebooks se ejecutaron de principio a fin sin errores antes de esta entrega.

Notebook 1 — Perceptrón, Red de Una Capa y Backpropagation

Clasificación binaria con funciones de activación Sigmoide y ReLU, comparando NumPy, TensorFlow/Keras y PyTorch.

0. Configuración del entorno

Este notebook está pensado para correr en **Google Colab**. Usa `numpy` y `matplotlib`, que ya vienen instaladas, además de `tensorflow` y `torch`, también disponibles por defecto en Colab (si hiciera falta, se instalan con `pip install tensorflow torch`).

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

print("NumPy:", np.__version__)
```

NumPy: 2.5.3

1. Funciones de activación: Sigmoide y ReLU

Empezamos por lo más básico: programar la Sigmoide y la ReLU, junto con sus derivadas (que vamos a necesitar más adelante para entrenar las redes con backpropagation), y graficarlas para ver de un vistazo cómo se comporta cada una.

```
def sigmoid(z):
    """Activación sigmoide: mapea R -> (0, 1). Satura para |z| grande (gradiente ~0)."""
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(a):
    """Derivada de la sigmoide en términos de su propia salida 'a'."""
    return a * (1 - a)

def relu(z):
    """Activación ReLU: max(0, z). No satura para z > 0."""
    return np.maximum(0, z)

def relu_derivative(z):
    """Derivada de ReLU: 1 si z > 0, 0 en caso contrario (evaluada sobre la pre-activación z)."""
    return (z > 0).astype(float)

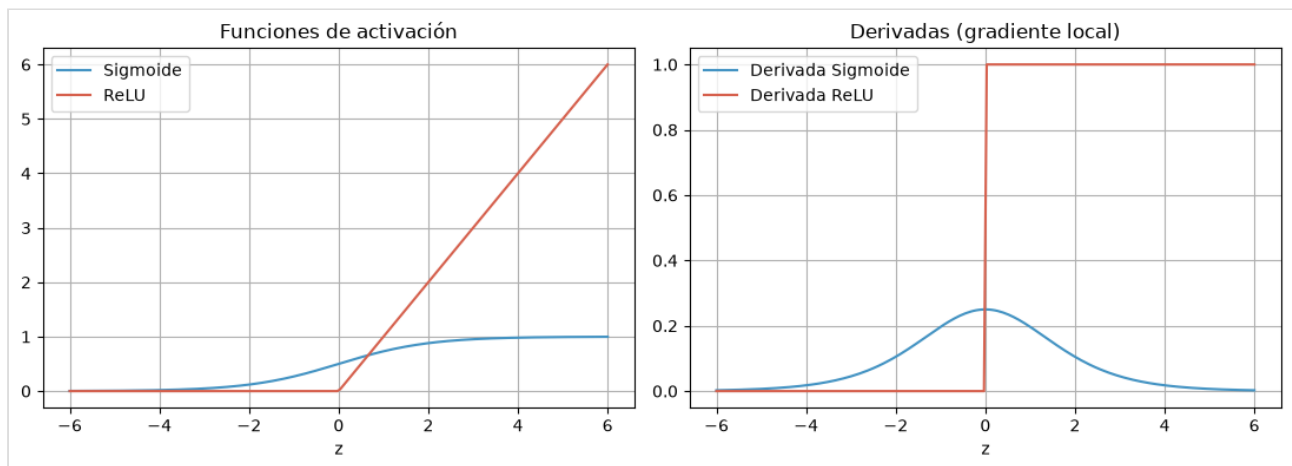
z = np.linspace(-6, 6, 200)

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(z, sigmoid(z), label="Sigmoide", color="#4393c3")
axes[0].plot(z, relu(z), label="ReLU", color="#d6604d")
axes[0].set_title("Funciones de activación")
axes[0].set_xlabel("z")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(z, sigmoid_derivative(sigmoid(z)), label="Derivada Sigmoide", color="#4393c3")
axes[1].plot(z, relu_derivative(z), label="Derivada ReLU", color="#d6604d")
axes[1].set_title("Derivadas (gradiente local)")
axes[1].set_xlabel("z")
axes[1].legend()
axes[1].grid(True)
```

```
plt.tight_layout()
plt.show()
```



En la gráfica de la derecha se nota bien la diferencia: la pendiente de la Sigmoide nunca pasa de 0.25, y para valores muy grandes o muy negativos prácticamente se aplana en cero — ahí es donde aparece el problema del desvanecimiento del gradiente. La ReLU, en cambio, mantiene una pendiente constante de 1 para todo valor positivo, así que el gradiente puede viajar por muchas capas sin irse debilitando.

2. Dataset de clasificación binaria (no linealmente separable)

Las compuertas lógicas de la entrega anterior eran un buen punto de partida, pero son un problema demasiado sencillo. Aquí usamos algo más realista: dos "semilunas" entrelazadas, generadas directamente con NumPy (sin depender de `scikit-learn`). Lo interesante de este dataset es que ninguna línea recta puede separar bien las dos clases — para lograrlo, hace falta una red con al menos una capa oculta.

```
def make_moons(n_samples=300, noise=0.15, seed=42):
    """Genera un dataset sintético de dos semilunas entrelazadas (clasificación binaria)."""
    rng = np.random.RandomState(seed)
    n1 = n_samples // 2
    n2 = n_samples - n1

    # Semiluna superior (clase 0)
    theta1 = np.linspace(0, np.pi, n1)
    x1 = np.stack([np.cos(theta1), np.sin(theta1)], axis=1)

    # Semiluna inferior, desplazada (clase 1)
    theta2 = np.linspace(0, np.pi, n2)
    x2 = np.stack([1 - np.cos(theta2), 1 - np.sin(theta2) - 0.5], axis=1)

    X = np.vstack([x1, x2])
    y = np.hstack([np.zeros(n1), np.ones(n2)])

    X += rng.randn(*X.shape) * noise # ruido gaussiano

    idx = rng.permutation(len(X))
    return X[idx], y[idx].reshape(-1, 1)

X, y = make_moons(n_samples=300, noise=0.15, seed=42)

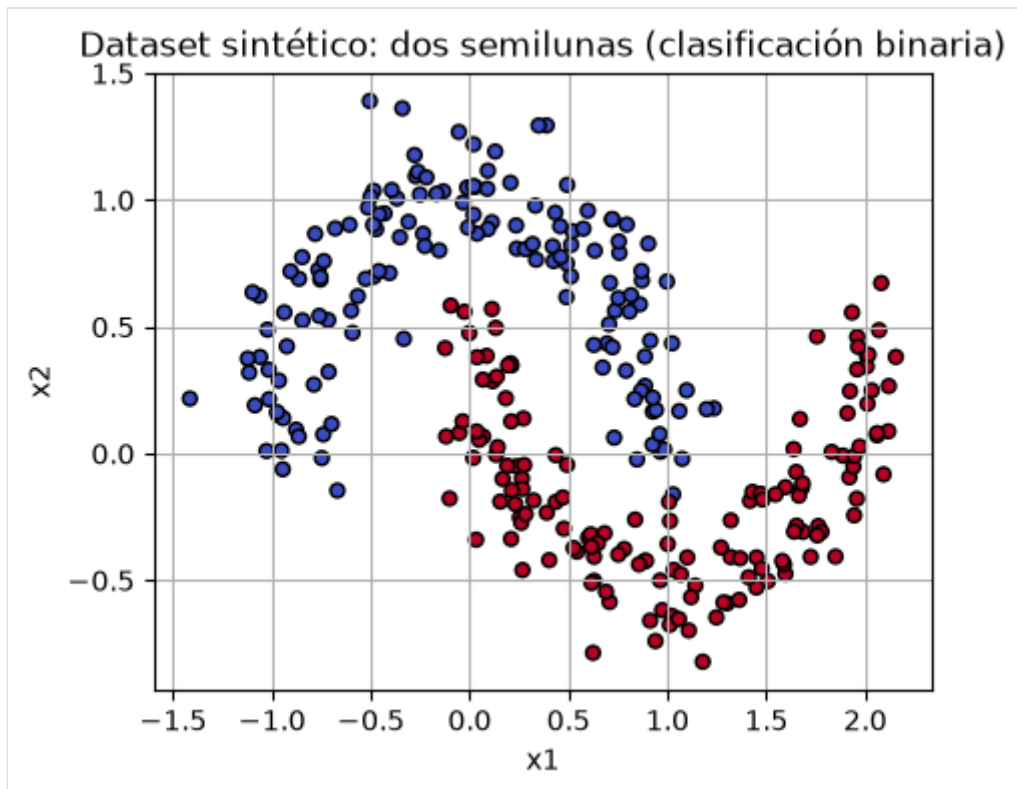
# División train/test (80/20) manual con NumPy
n_train = int(0.8 * len(X))
X_train, X_test = X[:n_train], X[n_train:]
y_train, y_test = y[:n_train], y[n_train:]

print("X_train:", X_train.shape, "| X_test:", X_test.shape)

plt.figure(figsize=(5, 4))
plt.scatter(X[:, 0], X[:, 1], c=y.flatten(), cmap="coolwarm", edgecolors="k", s=25)
plt.title("Dataset sintético: dos semilunas (clasificación binaria)")
plt.xlabel("x1")
plt.ylabel("x2")
```

```
plt.grid(True)
plt.show()
```

X_train: (240, 2) | X_test: (60, 2)



3. Perceptrón simple rediseñado

Volvemos al **perceptrón** de la entrega anterior, que entonces aprendía sin problema una compuerta lógica sencilla como AND. Ahora lo ponemos a prueba con el dataset de semilunas para ver, en la práctica, su principal limitación: un perceptrón solo puede dibujar una línea recta como frontera entre las dos clases, así que ante un problema como este — donde las clases están entrelazadas en forma de curva— no le va a ir bien.

```
def step_function(z):
    """Función de activación escalón: 1 si z >= 0, 0 en caso contrario."""
    return np.where(z >= 0, 1, 0)

class Perceptron:
    """
    Perceptrón simple con regla de aprendizaje clásica (Rosenblatt).
    Código base de la entrega anterior (Redes Neuronales Básicas), reutilizado aquí
    para contrastarlo con un problema no linealmente separable.
    """

    def __init__(self, n_inputs, learning_rate=0.1):
        self.weights = np.zeros(n_inputs)
        self.bias = 0.0
        self.lr = learning_rate

    def predict(self, X):
        z = X @ self.weights + self.bias
        return step_function(z)

    def fit(self, X, y, epochs=50):
        history = []
        for epoch in range(epochs):
            y_pred = self.predict(X)
            error = y - y_pred

            self.weights += self.lr * (X.T @ error)
```

```

        self.bias += self.lr * np.sum(error)

        loss = np.mean(np.abs(error))
        history.append(loss)
        if (epoch + 1) % 10 == 0:
            print(f"Época {epoch + 1:2d} | Error medio absoluto: {loss:.4f}")

        if loss == 0:
            print("Convergencia alcanzada.")
            break
    return history

def accuracy(self, X, y):
    preds = self.predict(X)
    return np.mean(preds == y.flatten()) * 100

perceptron = Perceptron(n_inputs=2, learning_rate=0.1)
history_perceptron = perceptron.fit(X_train, y_train.flatten(), epochs=50)

acc_perceptron_train = perceptron.accuracy(X_train, y_train)
acc_perceptron_test = perceptron.accuracy(X_test, y_test)
print(f"\nPerceptrón -> Accuracy train: {acc_perceptron_train:.1f}% | test: {acc_perceptron_test:.1f}%")

```

```

Época 10 | Error medio absoluto: 0.1375
Época 20 | Error medio absoluto: 0.1375
Época 30 | Error medio absoluto: 0.3125
Época 40 | Error medio absoluto: 0.1625
Época 50 | Error medio absoluto: 0.1583

```

```

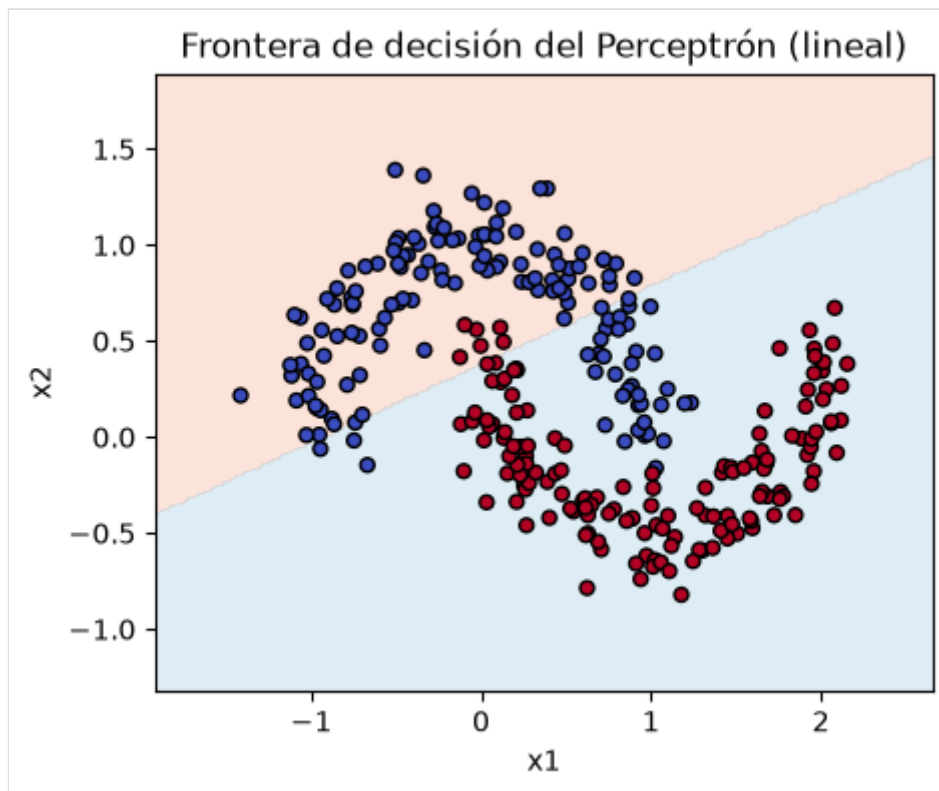
Perceptrón -> Accuracy train: 84.6% | test: 86.7%

```

```

plt.figure(figsize=(5, 4))
x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
Z = perceptron.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.3, levels=[-0.5, 0.5, 1.5], colors=["#f4a582", "#92c5de"])
plt.scatter(X[:, 0], X[:, 1], c=y.flatten(), cmap="coolwarm", edgecolors="k", s=25)
plt.title("Frontera de decisión del Perceptrón (lineal)")
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()

```



Como se ve en la gráfica, el perceptrón efectivamente solo logra trazar una línea recta, y eso no alcanza para separar bien las dos semilunas — su exactitud queda claramente por debajo de la que van a lograr más adelante los modelos con capa oculta. Es la limitación del perceptrón, ya no en teoría sino comprobada con un ejemplo concreto.

4. Red neuronal de una capa rediseñada (Sigmoide vs. ReLU)

También retomamos la **red neuronal de una capa** de la entrega anterior (sin capas ocultas), que en su momento aprendió la compuerta OR usando sigmoide. Ahora la ajustamos para que pueda usar tanto Sigmoide como ReLU, y la entrenamos con backpropagation sobre el mismo dataset de semilunas. El detalle importante aquí es que, aunque cambiemos la activación, esta red sigue siendo un clasificador lineal por dentro: no tener una capa oculta significa que no puede curvar su frontera de decisión, sin importar qué función de activación usemos.

```
class SingleLayerNN:
    """
    Red neuronal de una sola capa (sin capas ocultas), entrenada con backpropagation.
    Rediseño de la entrega anterior: ahora acepta Sigmoide o ReLU como activación,
    para comparar su efecto incluso en una arquitectura sin capa oculta.
    """

    def __init__(self, n_inputs, n_outputs, activation="sigmoid", learning_rate=0.5, seed=42):
        rng = np.random.RandomState(seed)
        self.W = rng.randn(n_inputs, n_outputs) * 0.1
        self.b = np.zeros((1, n_outputs))
        self.activation = activation
        self.lr = learning_rate

    def _activate(self, z):
        return relu(z) if self.activation == "relu" else sigmoid(z)

    def _activate_derivative(self, z, a):
        return relu_derivative(z) if self.activation == "relu" else sigmoid_derivative(a)

    def forward(self, X):
        self.z = X @ self.W + self.b
        self.a = self._activate(self.z)
        return self.a

    def backward(self, X, y):
        m = X.shape[0]
```

```

        error = self.a - y

        dz = error * self._activate_derivative(self.z, self.a)
        dW = (X.T @ dz) / m
        db = np.sum(dz, axis=0, keepdims=True) / m

        self.W -= self.lr * dW
        self.b -= self.lr * db

        return np.mean(error ** 2)

    def fit(self, X, y, epochs=1000, verbose_every=250):
        history = []
        for epoch in range(epochs):
            self.forward(X)
            loss = self.backward(X, y)
            history.append(loss)
            if (epoch + 1) % verbose_every == 0:
                print(f"[{self.activation.upper():7s}] Época {epoch + 1:4d} | Error cuadrático medio: {loss:.4f}")
        return history

    def predict(self, X, threshold=0.5):
        return (self.forward(X) >= threshold).astype(int)

    def accuracy(self, X, y):
        preds = self.predict(X)
        return np.mean(preds == y) * 100

single_layer_sigmoid = SingleLayerNN(n_inputs=2, n_outputs=1, activation="sigmoid", learning_rate=0.5, seed=42)
history_single_sigmoid = single_layer_sigmoid.fit(X_train, y_train, epochs=1000, verbose_every=250)

print()

single_layer_relu = SingleLayerNN(n_inputs=2, n_outputs=1, activation="relu", learning_rate=0.1, seed=42)
history_single_relu = single_layer_relu.fit(X_train, y_train, epochs=1000, verbose_every=250)

acc_single_sigmoid_train = single_layer_sigmoid.accuracy(X_train, y_train)
acc_single_sigmoid_test = single_layer_sigmoid.accuracy(X_test, y_test)
acc_single_relu_train = single_layer_relu.accuracy(X_train, y_train)
acc_single_relu_test = single_layer_relu.accuracy(X_test, y_test)

print(f"\nRed 1 capa (Sigmoide) -> Accuracy train: {acc_single_sigmoid_train:.1f}% | test: {acc_single_sigmoid_test:.1f}%")
print(f"Red 1 capa (ReLU) -> Accuracy train: {acc_single_relu_train:.1f}% | test: {acc_single_relu_test:.1f}%")

```

```

[SIGMOID] Época 250 | Error cuadrático medio: 0.1010
[SIGMOID] Época 500 | Error cuadrático medio: 0.0933
[SIGMOID] Época 750 | Error cuadrático medio: 0.0908
[SIGMOID] Época 1000 | Error cuadrático medio: 0.0898

[RELU ] Época 250 | Error cuadrático medio: 0.0912
[RELU ] Época 500 | Error cuadrático medio: 0.0911
[RELU ] Época 750 | Error cuadrático medio: 0.0911
[RELU ] Época 1000 | Error cuadrático medio: 0.0911

Red 1 capa (Sigmoide) -> Accuracy train: 87.9% | test: 88.3%
Red 1 capa (ReLU) -> Accuracy train: 88.3% | test: 86.7%

```

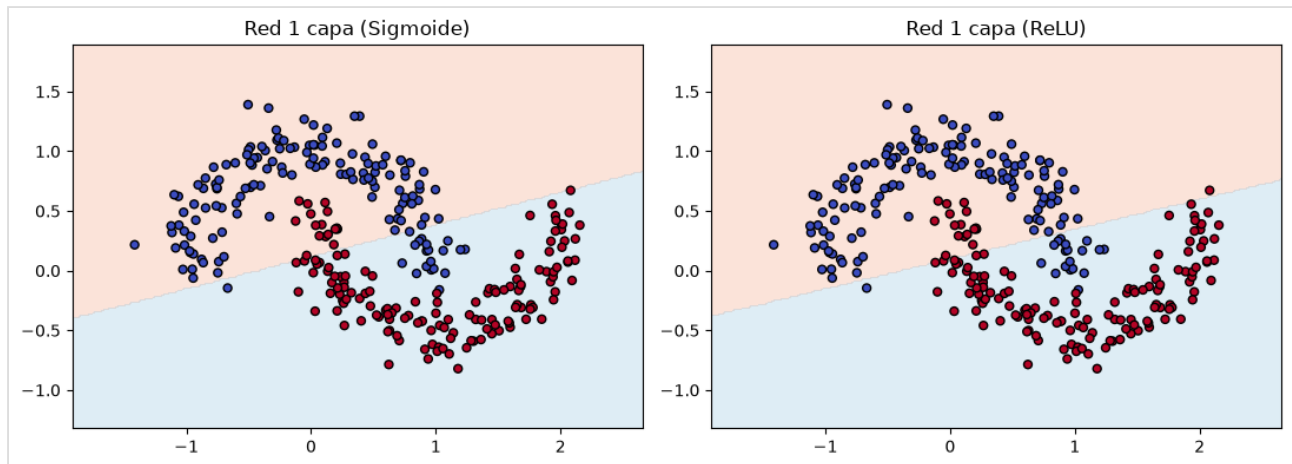
```

def plot_decision_boundary(predict_fn, X, y, title, ax):
    """Dibuja la frontera de decisión de un modelo sobre una malla de puntos (reutilizada en todo el notebook)."""
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
    grid = np.c_[xx.ravel(), yy.ravel()]
    Z = predict_fn(grid).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.3, levels=[-0.5, 0.5, 1.5], colors=["#f4a582", "#92c5de"])
    ax.scatter(X[:, 0], X[:, 1], c=y.flatten(), cmap="coolwarm", edgecolors="k", s=25)

```

```
ax.set_title(title)
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
plot_decision_boundary(single_layer_sigmoid.predict, X, y, "Red 1 capa (Sigmoide)", axes[0])
plot_decision_boundary(single_layer_relu.predict, X, y, "Red 1 capa (ReLU)", axes[1])
plt.tight_layout()
plt.show()
```



Tal como se esperaba, ni la Sigmoide ni la ReLU le dan a esta red la capacidad de curvar su frontera — sigue siendo una recta, y por eso su exactitud sobre las semilunas termina siendo parecida a la del perceptrón. Esto deja muy claro el punto que motiva la siguiente sección: para aprovechar de verdad la no linealidad de ReLU o Sigmoide, hace falta al menos una capa oculta que le dé a la red la posibilidad de combinar varias líneas y así aproximar una frontera curva.

5. MLP "vainilla" desde cero con backpropagation

Ahora sí armamos un perceptrón multicapa (MLP) con una capa oculta (que podemos configurar en Sigmoide o ReLU) y una capa de salida con sigmoide, pensada para clasificación binaria. Lo entrenamos con backpropagation y descenso de gradiente, usando como medida de error la **entropía cruzada binaria**:

[fórmula: $\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i) \right]$]

Esta forma de medir el error funciona mejor que el error cuadrático medio para este tipo de problema, porque castiga con más fuerza las predicciones que están muy seguras de sí mismas pero resultan equivocadas.

```
def binary_cross_entropy(y_true, y_pred, eps=1e-8):
    """Pérdida de entropía cruzada binaria (con recorte para evitar log(0))."""
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

class MLPBinaryClassifier:
    """
    MLP vainilla desde cero para clasificación binaria:
    capa de entrada -> capa oculta (activación configurable) -> capa de salida (sigmoide).
    Entrenado con backpropagation y descenso de gradiente por lotes.
    """

    def __init__(self, n_inputs, n_hidden, activation="relu", learning_rate=0.1, seed=42):
        rng = np.random.RandomState(seed)
        # Inicialización He para ReLU, Xavier para sigmoide (mejores prácticas de inicialización)
        scale = np.sqrt(2.0 / n_inputs) if activation == "relu" else np.sqrt(1.0 / n_inputs)
        self.W1 = rng.randn(n_inputs, n_hidden) * scale
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = rng.randn(n_hidden, 1) * np.sqrt(1.0 / n_hidden)
        self.b2 = np.zeros((1, 1))

        self.activation = activation
        self.lr = learning_rate

    def _activate(self, z):
        return relu(z) if self.activation == "relu" else sigmoid(z)
```

```

def _activate_derivative(self, z, a):
    return relu_derivative(z) if self.activation == "relu" else sigmoid_derivative(a)

def forward(self, X):
    self.z1 = X @ self.W1 + self.b1
    self.a1 = self._activate(self.z1)

    self.z2 = self.a1 @ self.W2 + self.b2
    self.a2 = sigmoid(self.z2) # salida siempre sigmoide (probabilidad de clase 1)
    return self.a2

def backward(self, X, y):
    m = X.shape[0]

    # --- Gradiente en la capa de salida (BCE + sigmoide se simplifica a: a2 - y) ---
    dz2 = self.a2 - y
    dW2 = (self.a1.T @ dz2) / m
    db2 = np.sum(dz2, axis=0, keepdims=True) / m

    # --- Propagación del error hacia la capa oculta ---
    error_hidden = dz2 @ self.W2.T
    dz1 = error_hidden * self._activate_derivative(self.z1, self.a1)
    dW1 = (X.T @ dz1) / m
    db1 = np.sum(dz1, axis=0, keepdims=True) / m

    # --- Actualización de parámetros ---
    self.W2 -= self.lr * dW2
    self.b2 -= self.lr * db2
    self.W1 -= self.lr * dW1
    self.b1 -= self.lr * db1

    return binary_cross_entropy(y, self.a2)

def fit(self, X, y, epochs=2000, verbose_every=400):
    history = []
    for epoch in range(epochs):
        self.forward(X)
        loss = self.backward(X, y)
        history.append(loss)
        if (epoch + 1) % verbose_every == 0:
            print(f"[{self.activation.upper():7s}] Época {epoch + 1:4d} | BCE: {loss:.4f}")
    return history

def predict(self, X, threshold=0.5):
    return (self.forward(X) >= threshold).astype(int)

def accuracy(self, X, y):
    preds = self.predict(X)
    return np.mean(preds == y) * 100

```

```

# Entrenamos dos versiones: una con activación Sigmoide y otra con ReLU en la capa oculta
mlp_sigmoid = MLPBinaryClassifier(n_inputs=2, n_hidden=8, activation="sigmoid", learning_rate=0.5, seed=42)
history_sigmoid = mlp_sigmoid.fit(X_train, y_train, epochs=2000, verbose_every=500)

print()

mlp_relu = MLPBinaryClassifier(n_inputs=2, n_hidden=8, activation="relu", learning_rate=0.5, seed=42)
history_relu = mlp_relu.fit(X_train, y_train, epochs=2000, verbose_every=500)

acc_sigmoid_train = mlp_sigmoid.accuracy(X_train, y_train)
acc_sigmoid_test = mlp_sigmoid.accuracy(X_test, y_test)
acc_relu_train = mlp_relu.accuracy(X_train, y_train)
acc_relu_test = mlp_relu.accuracy(X_test, y_test)

print(f"\nMLP (Sigmoide) -> Accuracy train: {acc_sigmoid_train:.1f}% | test: {acc_sigmoid_test:.1f}%")
print(f"MLP (ReLU) -> Accuracy train: {acc_relu_train:.1f}% | test: {acc_relu_test:.1f}%")

```

```

[SIGMOID] Época 500 | BCE: 0.2835
[SIGMOID] Época 1000 | BCE: 0.2812
[SIGMOID] Época 1500 | BCE: 0.2798
[SIGMOID] Época 2000 | BCE: 0.2789

```

```
[RELU ] Época 500 | BCE: 0.1541
[RELU ] Época 1000 | BCE: 0.0369
```

```
[RELU ] Época 1500 | BCE: 0.0220
[RELU ] Época 2000 | BCE: 0.0165
```

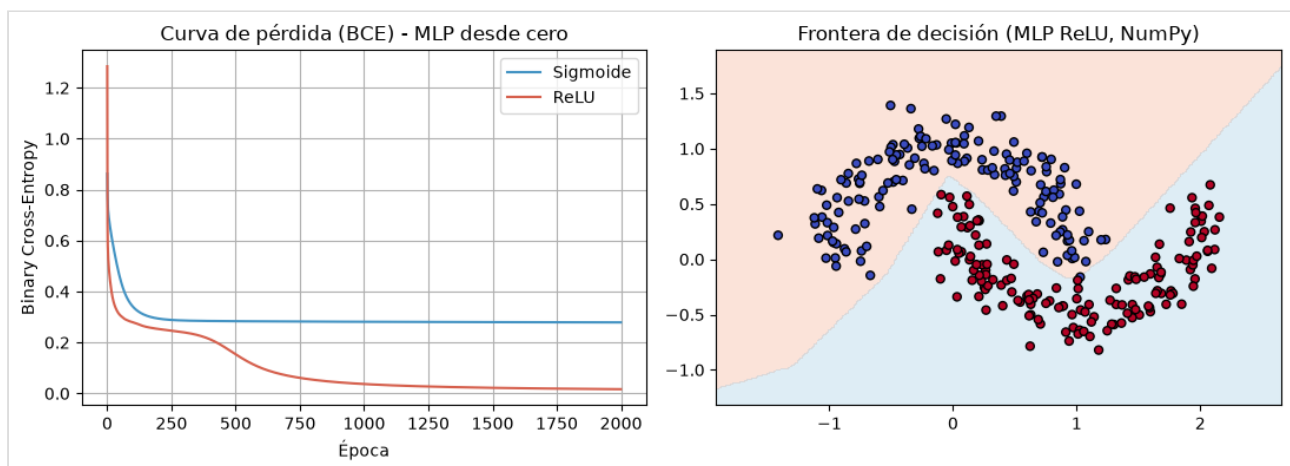
```
MLP (Sigmoide) -> Accuracy train: 88.3% | test: 86.7%
MLP (ReLU)      -> Accuracy train: 99.6% | test: 100.0%
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(history_sigmoide, label="Sigmoide", color="#4393c3")
axes[0].plot(history_relu, label="ReLU", color="#d6604d")
axes[0].set_title("Curva de pérdida (BCE) - MLP desde cero")
axes[0].set_xlabel("Época")
axes[0].set_ylabel("Binary Cross-Entropy")
axes[0].legend()
axes[0].grid(True)

plot_decision_boundary(mlp_relu.predict, X, y, "Frontera de decisión (MLP ReLU, NumPy)", axes[1])

plt.tight_layout()
plt.show()
```



Las dos versiones (Sigmoide y ReLU) terminan clasificando bien el dataset de semilunas, pero se nota que **ReLU aprende más rápido** — la pérdida baja mucho más deprisa en las primeras épocas, justamente porque su gradiente no se debilita como el de la sigmoide, y eso permite que cada actualización de los pesos sea más efectiva.

6. El mismo problema con TensorFlow + Keras

Ahora repetimos el mismo experimento de clasificación binaria, pero usando Keras. La arquitectura es un poco más profunda que la del MLP manual ($2 \rightarrow 16 \rightarrow 8 \rightarrow 1$, con ReLU en las capas ocultas y sigmoide en la salida), y esta vez entrenamos con **Adam** (una variante del descenso de gradiente que ajusta el paso de aprendizaje sobre la marcha) y en mini-lotes, que es como normalmente se entrena en la práctica.

```
import tensorflow as tf
from tensorflow import keras

tf.random.set_seed(42)

model_keras = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    keras.layers.Dense(16, activation="relu"), # primera capa oculta
    keras.layers.Dense(8, activation="relu"),  # segunda capa oculta
    keras.layers.Dense(1, activation="sigmoid") # capa de salida para clasificación binaria
])

model_keras.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
)  
  
model_keras.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	48
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 1)	9

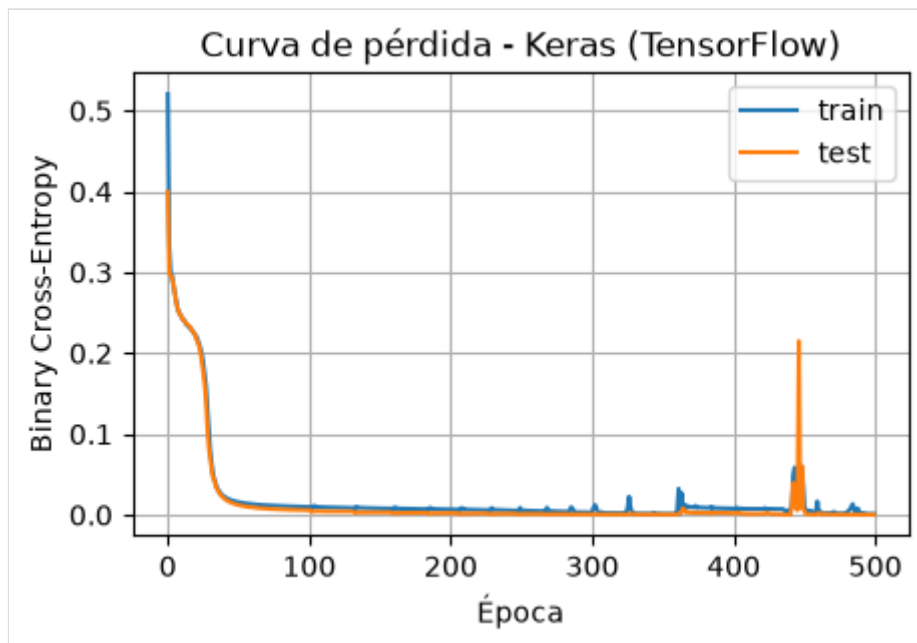
Total params: 193 (772.00 B)

Trainable params: 193 (772.00 B)

Non-trainable params: 0 (0.00 B)

```
history_keras = model_keras.fit(  
    X_train, y_train,  
    epochs=500,  
    batch_size=16,  
    verbose=0,  
    validation_data=(X_test, y_test)  
)  
  
loss_keras, acc_keras_train = model_keras.evaluate(X_train, y_train, verbose=0)  
loss_keras_test, acc_keras_test = model_keras.evaluate(X_test, y_test, verbose=0)  
  
print(f"Keras -> Accuracy train: {acc_keras_train*100:.1f}% | test: {acc_keras_test*100:.1f}%")  
  
plt.figure(figsize=(5, 3))  
plt.plot(history_keras.history["loss"], label="train")  
plt.plot(history_keras.history["val_loss"], label="test")  
plt.title("Curva de pérdida - Keras (TensorFlow)")  
plt.xlabel("Época")  
plt.ylabel("Binary Cross-Entropy")  
plt.legend()  
plt.grid(True)  
plt.show()
```

Keras -> Accuracy train: 100.0% | test: 100.0%



7. El mismo problema con PyTorch

Por último, replicamos esa misma arquitectura ($2 \rightarrow 16 \rightarrow 8 \rightarrow 1$) y el mismo esquema de entrenamiento (Adam, mini-lotes) pero con PyTorch. Aquí se nota bien la diferencia de estilo: mientras Keras resuelve casi todo con `.fit()`, en PyTorch tenemos que escribir explícitamente cada paso del entrenamiento.

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(42)

X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32)

train_loader = DataLoader(
    TensorDataset(X_train_t, y_train_t),
    batch_size=16,
    shuffle=True,
    generator=torch.Generator().manual_seed(42)
)

class TorchMLP(nn.Module):
    """Misma arquitectura que en Keras: 2 -> 16 (ReLU) -> 8 (ReLU) -> 1 (Sigmoide)."""
    def __init__(self, n_inputs=2, n_hidden1=16, n_hidden2=8):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_inputs, n_hidden1),
            nn.ReLU(),
            nn.Linear(n_hidden1, n_hidden2),
            nn.ReLU(),
            nn.Linear(n_hidden2, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

model_torch = TorchMLP()
```

```
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model_torch.parameters(), lr=0.01)
```

```
history_torch = []

for epoch in range(500):
    for xb, yb in train_loader:
        optimizer.zero_grad()
        outputs = model_torch(xb)
        loss = criterion(outputs, yb)
        loss.backward()      # backpropagation automático (autograd)
        optimizer.step()     # actualización de pesos (descenso de gradiente)

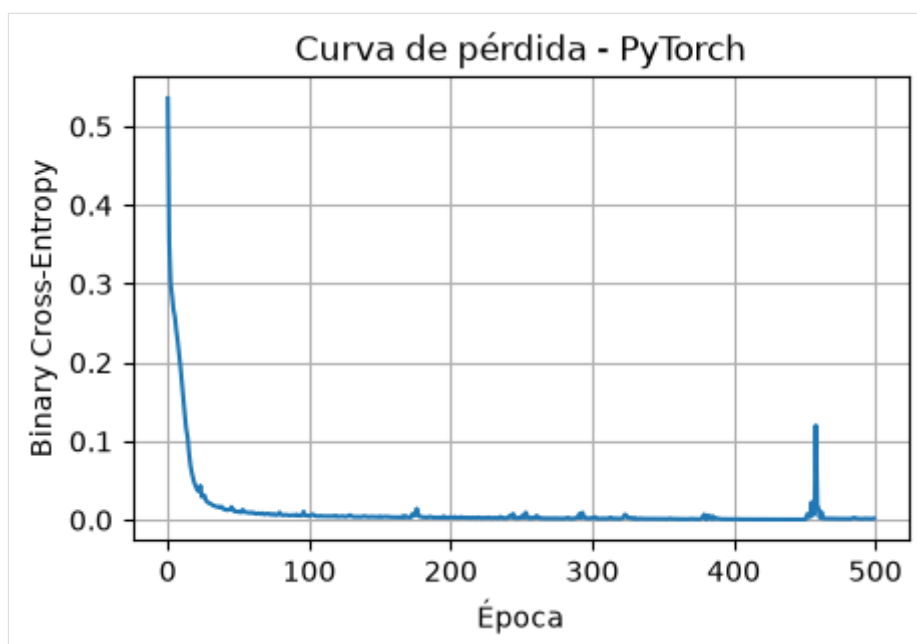
    with torch.no_grad():
        epoch_loss = criterion(model_torch(X_train_t), y_train_t).item()
        history_torch.append(epoch_loss)

    with torch.no_grad():
        preds_train = (model_torch(X_train_t) >= 0.5).float()
        preds_test = (model_torch(X_test_t) >= 0.5).float()
        acc_torch_train = (preds_train == y_train_t).float().mean().item() * 100
        acc_torch_test = (preds_test == y_test_t).float().mean().item() * 100

print(f"PyTorch -> Accuracy train: {acc_torch_train:.1f}% | test: {acc_torch_test:.1f}%")

plt.figure(figsize=(5, 3))
plt.plot(history_torch)
plt.title("Curva de pérdida - PyTorch")
plt.xlabel("Época")
plt.ylabel("Binary Cross-Entropy")
plt.grid(True)
plt.show()
```

PyTorch -> Accuracy train: 100.0% | test: 100.0%



8. Comparación de resultados entre todas las implementaciones

Juntamos aquí los resultados de los seis modelos que entrenamos sobre el mismo dataset de semilunas: el perceptrón, la red de una capa (con Sigmoid y con ReLU), el MLP manual (con Sigmoid y con ReLU), y las versiones del MLP en Keras y PyTorch (ambas con ReLU).

```
print("Resumen de exactitud (accuracy) por implementación – dataset de semilunas\n")
print(f"{'Implementación':<28}{'Train':>10}{'Test':>10}")
print("-" * 48)
print(f"{'Perceptrón simple':<28}{acc_perceptron_train:>9.1f}%{acc_perceptron_test:>9.1f}%")
```

```
print(f"{'Red 1 capa (Sigmoide)':<28}{acc_single_sigmoid_train:>9.1f}%{acc_single_sigmoid_test:>9.1f}%")
print(f"{'Red 1 capa (ReLU)':<28}{acc_single_relu_train:>9.1f}%{acc_single_relu_test:>9.1f}%")
print(f"{'MLP desde cero (Sigmoide)':<28}{acc_sigmoid_train:>9.1f}%{acc_sigmoid_test:>9.1f}%")
print(f"{'MLP desde cero (ReLU)':<28}{acc_relu_train:>9.1f}%{acc_relu_test:>9.1f}%")
print(f"{'TensorFlow + Keras (ReLU)':<28}{acc_keras_train*100:>9.1f}%{acc_keras_test*100:>9.1f}%")
print(f"{'PyTorch (ReLU)':<28}{acc_torch_train:>9.1f}%{acc_torch_test:>9.1f}%")
```

Resumen de exactitud (accuracy) por implementación – dataset de semilunas

Implementación	Train	Test
Perceptrón simple	84.6%	86.7%
Red 1 capa (Sigmoide)	87.9%	88.3%
Red 1 capa (ReLU)	88.3%	86.7%
MLP desde cero (Sigmoide)	88.3%	86.7%
MLP desde cero (ReLU)	99.6%	100.0%
TensorFlow + Keras (ReLU)	100.0%	100.0%
PyTorch (ReLU)	100.0%	100.0%

9. Conclusiones

- El perceptrón y la red de una capa, al ser modelos lineales, solo pueden trazar fronteras rectas. Por eso su exactitud sobre las semilunas se queda bastante por debajo de la que logran los modelos con capa oculta — esto confirma en la práctica algo que ya sabíamos en teoría, y que en la entrega anterior era menos evidente porque las compuertas lógicas eran un problema más sencillo.
- Comparando ReLU contra Sigmoide en la capa oculta del MLP, ReLU aprende más rápido porque su gradiente no se debilita con valores positivos, evitando el desvanecimiento del gradiente que sí afecta a la sigmoide. Por eso ReLU se volvió la opción estándar para capas ocultas en arquitecturas más profundas, mientras que la sigmoide sigue siendo útil en la capa de salida cuando queremos interpretar el resultado como una probabilidad.
- Backpropagation es exactamente el mismo algoritmo tanto si lo programamos a mano con NumPy (en el perceptrón, la red de una capa y el MLP) como si lo dejamos en manos de Keras o PyTorch — la diferencia es que estos frameworks calculan los gradientes automáticamente (lo que se conoce como autograd), así que uno no tiene que derivar nada a mano.
- Keras permite armar y entrenar un modelo con muy poco código gracias a su estilo declarativo (`Sequential`, `.compile()`, `.fit()`).
- PyTorch pide escribir el ciclo de entrenamiento paso a paso (calcular la salida, medir el error, propagar el gradiente hacia atrás, actualizar los pesos), lo que implica algo más de código pero también más control sobre lo que está pasando.
- Al final, el MLP manual, Keras y PyTorch (los tres usando ReLU) llegan al mismo resultado (100% de exactitud) sobre las semilunas. Eso confirma que la base matemática es la misma en los tres casos — lo que cambia es solo la herramienta con la que se expresa.

Notebook 2 — Clasificación Multiclase con MNIST

Reconocimiento de dígitos con redes densas (sin CNN), implementado en TensorFlow/Keras y PyTorch.

0. Configuración del entorno y carga del dataset MNIST

Este notebook está pensado para correr en **Google Colab**. MNIST se descarga solo, usando las utilidades de `tensorflow.keras.datasets` para la parte de Keras y de `torchvision.datasets` para la parte de PyTorch.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

np.random.seed(42)
tf.random.set_seed(42)

print("NumPy:", np.__version__)
print("TensorFlow:", tf.__version__)
```

```
NumPy: 2.5.3
TensorFlow: 2.21.0
```

```
# Carga del dataset MNIST predefinido (incluido en Keras)
(X_train_full, y_train_full), (X_test, y_test) = keras.datasets.mnist.load_data()

print("Imágenes de entrenamiento:", X_train_full.shape)
print("Etiquetas de entrenamiento:", y_train_full.shape)
print("Imágenes de prueba:", X_test.shape)
print("Rango de píxeles:", X_train_full.min(), "-", X_train_full.max())
print("Clases:", np.unique(y_train_full))
```

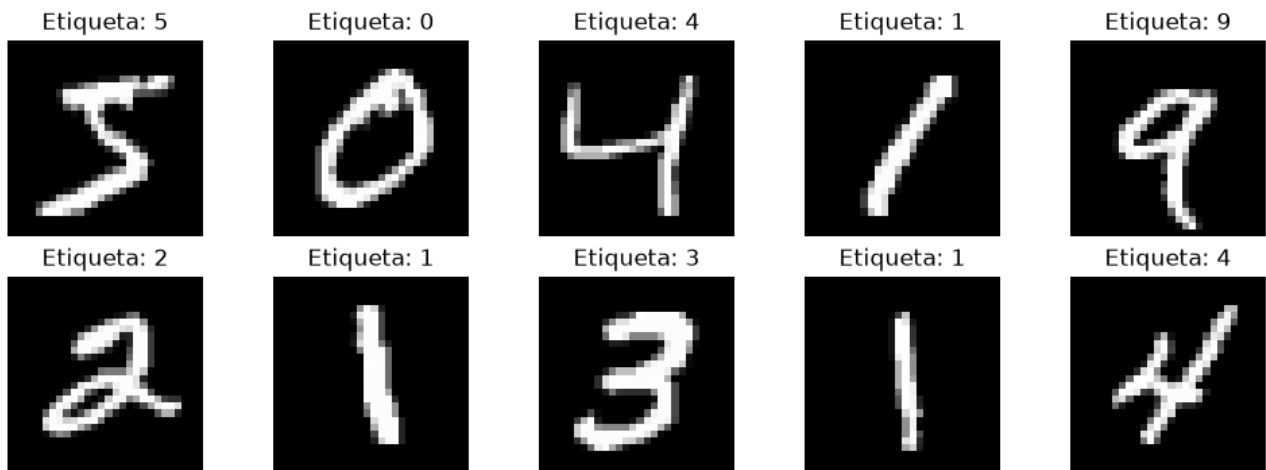
```
Imágenes de entrenamiento: (60000, 28, 28)
Etiquetas de entrenamiento: (60000,)
Imágenes de prueba: (10000, 28, 28)
Rango de píxeles: 0 - 255
Clases: [0 1 2 3 4 5 6 7 8 9]
```

1. Exploración y preprocesamiento de los datos

Primero echamos un vistazo a algunas imágenes del dataset, y después las preparamos para la red: llevamos los valores de los píxeles al rango [0, 1] (en vez de 0-255) y convertimos cada imagen de 28×28 en un vector de 784 números, ya que una red densa no entiende de "filas y columnas" como sí lo haría una CNN — solo recibe una lista larga de números.

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train_full[i], cmap="gray")
    ax.set_title(f"Etiqueta: {y_train_full[i]}")
    ax.axis("off")
plt.suptitle("Muestras del dataset MNIST")
plt.tight_layout()
plt.show()
```

Muestras del dataset MNIST



```
# Normalización: píxeles de [0, 255] a [0, 1] (acelera y estabiliza el entrenamiento)
X_train_full = X_train_full.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Aplanado: (N, 28, 28) -> (N, 784), ya que usamos capas densas, no convolucionales
X_train_full_flat = X_train_full.reshape(-1, 28 * 28)
X_test_flat = X_test.reshape(-1, 28 * 28)

# Separamos un conjunto de validación a partir del set de entrenamiento
n_val = 5000
X_val_flat, X_train_flat = X_train_full_flat[:n_val], X_train_full_flat[n_val:]
y_val, y_train = y_train_full[:n_val], y_train_full[n_val:]

print("Train:", X_train_flat.shape, "| Val:", X_val_flat.shape, "| Test:", X_test_flat.shape)
```

Train: (55000, 784) | Val: (5000, 784) | Test: (10000, 784)

2. Clasificación multiclase con TensorFlow + Keras

Armamos una red con la arquitectura **784 → 128 (ReLU) → 64 (ReLU) → 10 (Softmax)**, sin capas convolucionales. Como las etiquetas vienen como números enteros (0, 1, 2... 9) y no como vectores one-hot, usamos `sparse_categorical_crossentropy` como función de pérdida, que está pensada justo para ese caso.

```
model_keras_mnist = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(10, activation="softmax") # 10 clases, una por dígito
])

model_keras_mnist.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model_keras_mnist.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256

dense_2 (Dense)	(None, 10)	650
-----------------	------------	-----

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

```
history_keras_mnist = model_keras_mnist.fit(
    X_train_flat, y_train,
    epochs=10,
    batch_size=128,
    validation_data=(X_val_flat, y_val),
    verbose=1
)
```

Epoch 1/10

1/430 ————— 5:45 805ms/step - accuracy: 0.0625 - loss: 2.3767

24/430 ————— 0s 2ms/step - accuracy: 0.5983 - loss: 1.5359

46/430 ————— 0s 2ms/step - accuracy: 0.7135 - loss: 1.0846

70/430 ————— 0s 2ms/step - accuracy: 0.7731 - loss: 0.8556

91/430 ————— 0s 2ms/step - accuracy: 0.8000 - loss: 0.7414

112/430 ————— 0s 2ms/step - accuracy: 0.8171 - loss: 0.6715

134/430 ————— 0s 2ms/step - accuracy: 0.8338 - loss: 0.6095

146/430 ————— 0s 2ms/step - accuracy: 0.8397 - loss: 0.5860

163/430 ————— 0s 3ms/step - accuracy: 0.8480 - loss: 0.5524

183/430 ————— 0s 3ms/step - accuracy: 0.8570 - loss: 0.5188

200/430 ————— 0s 3ms/step - accuracy: 0.8630 - loss: 0.4951

217/430 ————— 0s 3ms/step - accuracy: 0.8681 - loss: 0.4740

234/430 ————— 0s 3ms/step - accuracy: 0.8724 - loss: 0.4572

251/430 ————— 0s 3ms/step - accuracy: 0.8765 - loss: 0.4409

269/430 ————— 0s 3ms/step - accuracy: 0.8801 - loss: 0.4273

286/430 ————— 0s 3ms/step - accuracy: 0.8834 - loss: 0.4149

302/430 ————— 0s 3ms/step - accuracy: 0.8863 - loss: 0.4041

319/430 0s 3ms/step - accuracy: 0.8891 - loss: 0.3940

335/430 0s 3ms/step - accuracy: 0.8920 - loss: 0.3849

351/430 0s 3ms/step - accuracy: 0.8943 - loss: 0.3764

368/430 0s 3ms/step - accuracy: 0.8968 - loss: 0.3671

387/430 0s 3ms/step - accuracy: 0.8986 - loss: 0.3596

405/430 0s 3ms/step - accuracy: 0.9005 - loss: 0.3527

425/430 0s 3ms/step - accuracy: 0.9026 - loss: 0.3455

430/430 2s 3ms/step - accuracy: 0.9032 - loss: 0.3436 - val_accuracy: 0.9528 - val_loss: 0.1619

Epoch 2/10

1/430 11s 27ms/step - accuracy: 0.9844 - loss: 0.1319

18/430 1s 3ms/step - accuracy: 0.9523 - loss: 0.1626

34/430 1s 3ms/step - accuracy: 0.9536 - loss: 0.1600

51/430 1s 3ms/step - accuracy: 0.9539 - loss: 0.1597

67/430 1s 3ms/step - accuracy: 0.9539 - loss: 0.1598

85/430 1s 3ms/step - accuracy: 0.9544 - loss: 0.1567

109/430 0s 3ms/step - accuracy: 0.9534 - loss: 0.1563

134/430 0s 3ms/step - accuracy: 0.9545 - loss: 0.1554

159/430 0s 3ms/step - accuracy: 0.9539 - loss: 0.1548

183/430 0s 3ms/step - accuracy: 0.9550 - loss: 0.1500

206/430 0s 2ms/step - accuracy: 0.9560 - loss: 0.1470

232/430 0s 2ms/step - accuracy: 0.9567 - loss: 0.1443

258/430 0s 2ms/step - accuracy: 0.9575 - loss: 0.1420

283/430 0s 2ms/step - accuracy: 0.9577 - loss: 0.1408

309/430 0s 2ms/step - accuracy: 0.9579 - loss: 0.1402

332/430 0s 2ms/step - accuracy: 0.9581 - loss: 0.1387

356/430 0s 2ms/step - accuracy: 0.9585 - loss: 0.1377

379/430 0s 2ms/step - accuracy: 0.9590 - loss: 0.1367

404/430 0s 2ms/step - accuracy: 0.9590 - loss: 0.1364

427/430 0s 2ms/step - accuracy: 0.9592 - loss: 0.1356

430/430 1s 3ms/step - accuracy: 0.9593 - loss: 0.1354 - val_accuracy: 0.9680 - val_loss: 0.1134

Epoch 3/10

1/430 9s 22ms/step - accuracy: 0.9766 - loss: 0.0741

26/430 0s 2ms/step - accuracy: 0.9715 - loss: 0.1042

46/430 0s 2ms/step - accuracy: 0.9704 - loss: 0.1049

62/430 0s 3ms/step - accuracy: 0.9711 - loss: 0.1035

80/430 0s 3ms/step - accuracy: 0.9711 - loss: 0.1040

100/430 0s 3ms/step - accuracy: 0.9705 - loss: 0.1033

121/430 0s 3ms/step - accuracy: 0.9697 - loss: 0.1059

144/430 0s 3ms/step - accuracy: 0.9697 - loss: 0.1068

166/430 0s 3ms/step - accuracy: 0.9704 - loss: 0.1037

189/430 0s 2ms/step - accuracy: 0.9710 - loss: 0.1013

211/430 0s 2ms/step - accuracy: 0.9716 - loss: 0.0995

234/430 0s 2ms/step - accuracy: 0.9718 - loss: 0.0982

258/430 0s 2ms/step - accuracy: 0.9719 - loss: 0.0969

284/430 0s 2ms/step - accuracy: 0.9720 - loss: 0.0961

310/430 0s 2ms/step - accuracy: 0.9715 - loss: 0.0964

334/430 0s 2ms/step - accuracy: 0.9716 - loss: 0.0954

355/430 0s 2ms/step - accuracy: 0.9717 - loss: 0.0951

362/430 0s 2ms/step - accuracy: 0.9718 - loss: 0.0948

370/430 0s 3ms/step - accuracy: 0.9719 - loss: 0.0944

379/430 0s 3ms/step - accuracy: 0.9721 - loss: 0.0944

393/430 0s 3ms/step - accuracy: 0.9721 - loss: 0.0944

405/430 0s 3ms/step - accuracy: 0.9719 - loss: 0.0947

418/430 0s 3ms/step - accuracy: 0.9720 - loss: 0.0944

430/430 1s 3ms/step - accuracy: 0.9721 - loss: 0.0942 - val_accuracy: 0.9710 - val_loss: 0.0940

Epoch 4/10

1/430 10s 24ms/step - accuracy: 0.9844 - loss: 0.0534

25/430 0s 2ms/step - accuracy: 0.9784 - loss: 0.0831

50/430 0s 2ms/step - accuracy: 0.9781 - loss: 0.0799

78/430 0s 2ms/step - accuracy: 0.9785 - loss: 0.0787

95/430 0s 2ms/step - accuracy: 0.9783 - loss: 0.0774

111/430 0s 2ms/step - accuracy: 0.9780 - loss: 0.0794

126/430 0s 2ms/step - accuracy: 0.9781 - loss: 0.0796

145/430 0s 2ms/step - accuracy: 0.9777 - loss: 0.0807

164/430 0s 3ms/step - accuracy: 0.9781 - loss: 0.0787

186/430 0s 2ms/step - accuracy: 0.9789 - loss: 0.0762

209/430 0s 2ms/step - accuracy: 0.9790 - loss: 0.0750

227/430 0s 2ms/step - accuracy: 0.9792 - loss: 0.0740

250/430 0s 2ms/step - accuracy: 0.9793 - loss: 0.0730

267/430 0s 3ms/step - accuracy: 0.9793 - loss: 0.0733

284/430 0s 3ms/step - accuracy: 0.9793 - loss: 0.0726

302/430 0s 3ms/step - accuracy: 0.9792 - loss: 0.0724

323/430 0s 3ms/step - accuracy: 0.9790 - loss: 0.0722

342/430 0s 3ms/step - accuracy: 0.9790 - loss: 0.0718

365/430 0s 3ms/step - accuracy: 0.9790 - loss: 0.0715

385/430 0s 3ms/step - accuracy: 0.9791 - loss: 0.0712

405/430 0s 3ms/step - accuracy: 0.9788 - loss: 0.0716

430/430 0s 3ms/step - accuracy: 0.9789 - loss: 0.0713

430/430 1s 3ms/step - accuracy: 0.9789 - loss: 0.0713 - val_accuracy: 0.9750 - val_loss: 0.0829

Epoch 5/10

1/430 10s 25ms/step - accuracy: 0.9844 - loss: 0.0459

22/430 1s 2ms/step - accuracy: 0.9812 - loss: 0.0646

36/430 1s 3ms/step - accuracy: 0.9829 - loss: 0.0634

48/430 1s 3ms/step - accuracy: 0.9826 - loss: 0.0648

58/430 1s 4ms/step - accuracy: 0.9833 - loss: 0.0622

72/430 1s 4ms/step - accuracy: 0.9833 - loss: 0.0619

79/430 1s 4ms/step - accuracy: 0.9834 - loss: 0.0618

84/430 1s 4ms/step - accuracy: 0.9834 - loss: 0.0608

89/430 1s 5ms/step - accuracy: 0.9838 - loss: 0.0606

97/430 1s 5ms/step - accuracy: 0.9837 - loss: 0.0602

118/430 1s 5ms/step - accuracy: 0.9829 - loss: 0.0616

145/430 1s 4ms/step - accuracy: 0.9822 - loss: 0.0630

165/430 1s 4ms/step - accuracy: 0.9829 - loss: 0.0610

184/430 0s 4ms/step - accuracy: 0.9833 - loss: 0.0594

200/430 0s 4ms/step - accuracy: 0.9834 - loss: 0.0587

217/430 0s 4ms/step - accuracy: 0.9835 - loss: 0.0583

236/430 0s 4ms/step - accuracy: 0.9838 - loss: 0.0575

256/430 0s 4ms/step - accuracy: 0.9839 - loss: 0.0570

277/430 0s 3ms/step - accuracy: 0.9838 - loss: 0.0567

292/430 0s 3ms/step - accuracy: 0.9838 - loss: 0.0563

300/430 0s 4ms/step - accuracy: 0.9838 - loss: 0.0565

309/430 0s 4ms/step - accuracy: 0.9837 - loss: 0.0567

321/430 0s 4ms/step - accuracy: 0.9836 - loss: 0.0563

334/430 0s 4ms/step - accuracy: 0.9837 - loss: 0.0559

364/430 0s 3ms/step - accuracy: 0.9839 - loss: 0.0553

389/430 0s 3ms/step - accuracy: 0.9840 - loss: 0.0553

413/430 0s 3ms/step - accuracy: 0.9839 - loss: 0.0553

430/430 2s 4ms/step - accuracy: 0.9838 - loss: 0.0553 - val_accuracy: 0.9752 - val_loss: 0.0815

Epoch 6/10

1/430 10s 24ms/step - accuracy: 0.9844 - loss: 0.0375

22/430 1s 2ms/step - accuracy: 0.9840 - loss: 0.0544

43/430 0s 2ms/step - accuracy: 0.9858 - loss: 0.0537

68/430 0s 2ms/step - accuracy: 0.9860 - loss: 0.0515

89/430 0s 2ms/step - accuracy: 0.9869 - loss: 0.0492

103/430 0s 3ms/step - accuracy: 0.9866 - loss: 0.0487

118/430	0s 3ms/step	accuracy: 0.9863	loss: 0.0494
135/430	0s 3ms/step	accuracy: 0.9862	loss: 0.0497
156/430	0s 3ms/step	accuracy: 0.9862	loss: 0.0492
172/430	0s 3ms/step	accuracy: 0.9865	loss: 0.0485
190/430	0s 3ms/step	accuracy: 0.9870	loss: 0.0467
209/430	0s 3ms/step	accuracy: 0.9871	loss: 0.0463
224/430	0s 3ms/step	accuracy: 0.9872	loss: 0.0459
232/430	0s 3ms/step	accuracy: 0.9872	loss: 0.0458
243/430	0s 3ms/step	accuracy: 0.9874	loss: 0.0451
254/430	0s 3ms/step	accuracy: 0.9874	loss: 0.0450
267/430	0s 3ms/step	accuracy: 0.9873	loss: 0.0452
279/430	0s 3ms/step	accuracy: 0.9874	loss: 0.0449
292/430	0s 3ms/step	accuracy: 0.9874	loss: 0.0446
305/430	0s 3ms/step	accuracy: 0.9873	loss: 0.0451
318/430	0s 3ms/step	accuracy: 0.9873	loss: 0.0447
336/430	0s 3ms/step	accuracy: 0.9874	loss: 0.0443
356/430	0s 3ms/step	accuracy: 0.9875	loss: 0.0440
375/430	0s 3ms/step	accuracy: 0.9876	loss: 0.0435
395/430	0s 3ms/step	accuracy: 0.9873	loss: 0.0442
414/430	0s 3ms/step	accuracy: 0.9873	loss: 0.0439
430/430	1s 3ms/step	accuracy: 0.9873	loss: 0.0439 - val_accuracy: 0.9742 - val_loss: 0.0886
Epoch 7/10			
1/430	10s 25ms/step	accuracy: 0.9922	loss: 0.0304

17/430 1s 4ms/step - accuracy: 0.9885 - loss: 0.0439

34/430 1s 3ms/step - accuracy: 0.9887 - loss: 0.0454

53/430 1s 3ms/step - accuracy: 0.9891 - loss: 0.0434

72/430 1s 3ms/step - accuracy: 0.9895 - loss: 0.0410

94/430 0s 3ms/step - accuracy: 0.9902 - loss: 0.0392

113/430 0s 3ms/step - accuracy: 0.9896 - loss: 0.0396

134/430 0s 3ms/step - accuracy: 0.9894 - loss: 0.0396

152/430 0s 3ms/step - accuracy: 0.9895 - loss: 0.0395

170/430 0s 3ms/step - accuracy: 0.9899 - loss: 0.0383

188/430 0s 3ms/step - accuracy: 0.9901 - loss: 0.0373

206/430 0s 3ms/step - accuracy: 0.9899 - loss: 0.0372

226/430 0s 3ms/step - accuracy: 0.9899 - loss: 0.0367

244/430 0s 3ms/step - accuracy: 0.9901 - loss: 0.0361

264/430 0s 3ms/step - accuracy: 0.9900 - loss: 0.0362

284/430 0s 3ms/step - accuracy: 0.9901 - loss: 0.0359

305/430 0s 3ms/step - accuracy: 0.9900 - loss: 0.0365

326/430 0s 3ms/step - accuracy: 0.9901 - loss: 0.0360

347/430 0s 3ms/step - accuracy: 0.9901 - loss: 0.0357

369/430 0s 3ms/step - accuracy: 0.9903 - loss: 0.0351

378/430 0s 3ms/step - accuracy: 0.9902 - loss: 0.0353

394/430 0s 3ms/step - accuracy: 0.9902 - loss: 0.0355

409/430 0s 3ms/step - accuracy: 0.9902 - loss: 0.0354

426/430 0s 3ms/step - accuracy: 0.9902 - loss: 0.0353

```

430/430 _____ 1s 3ms/step - accuracy: 0.9902 - loss: 0.0353 - val_accuracy: 0.9728 - val_loss: 0.0951

```

Epoch 8/10

1/430 ————— 10s 24ms/step - accuracy: 0.9922 - loss: 0.0298

22/430 ————— 0s 2ms/step - accuracy: 0.9925 - loss: 0.0360

```
██████████████████████████████████████████████████████████████████████████
47/430 ██████████ 0s 2ms/step - accuracy: 0.9905 - loss: 0.0397
```

```

67/430 ————— 0s 2ms/step - accuracy: 0.9915 - loss: 0.0360

```

```

88/430 ————— 0s 2ms/step - accuracy: 0.9919 - loss: 0.0341

```

```

112/430 ————— 0s 2ms/step - accuracy: 0.9911 - loss: 0.0335

```

```

131/430 ————— 0s 2ms/step - accuracy: 0.9911 - loss: 0.0327

```

```

148/430 ————— 0s 2ms/step - accuracy: 0.9913 - loss: 0.0325

```

```

163/430 ————— 0s 3ms/step - accuracy: 0.9912 - loss: 0.0317

```

```

178/430 ————— 0s 3ms/step - accuracy: 0.9917 - loss: 0.0309

```

```

193/430 ————— 0s 3ms/step - accuracy: 0.9916 - loss: 0.0308

```

```

208/430 ————— 0s 3ms/step - accuracy: 0.9915 - loss: 0.0305

```

```

223/430 ————— 0s 3ms/step - accuracy: 0.9915 - loss: 0.0303

```

```

██████████████████████████████████████████████████████████████████████████
247/430 ██████████ 0s 3ms/step - accuracy: 0.9916 - loss: 0.0296

```

```

██████████████████████████████████████████████████████████████████████████
268/430 ██████████ 0s 3ms/step - accuracy: 0.9915 - loss: 0.0300

```

```

291/430 ————— 0s 3ms/step - accuracy: 0.9916 - loss: 0.0296

```

```

316/430 ————— 0s 3ms/step - accuracy: 0.9914 - loss: 0.0299

```

```
██████████████████████████████████████████████████████████████████████████
342/430 ██████████ 0s 3ms/step - accuracy: 0.9914 - loss: 0.0296
```

```

366/430 _____ 0s 3ms/step - accuracy: 0.9916 - loss: 0.0291

```

```
██████████████████████████████████████████████████████████████████████████
387/430 ██████████ 0s 3ms/step - accuracy: 0.9917 - loss: 0.0288
```

```

410/430 ————— 0s 2ms/step - accuracy: 0.9919 - loss: 0.0288

```

```

430/430 _____ 1s 3ms/step - accuracy: 0.9919 - loss: 0.0287 - val_accuracy: 0.9690 - val_loss: 0.1059

```

Epoch 9/10	
1/430	13s 31ms/step - accuracy: 0.9844 - loss: 0.0358
14/430	1s 4ms/step - accuracy: 0.9900 - loss: 0.0314
29/430	1s 4ms/step - accuracy: 0.9895 - loss: 0.0358
46/430	1s 4ms/step - accuracy: 0.9900 - loss: 0.0360
66/430	1s 3ms/step - accuracy: 0.9912 - loss: 0.0323
86/430	1s 3ms/step - accuracy: 0.9925 - loss: 0.0294
106/430	0s 3ms/step - accuracy: 0.9926 - loss: 0.0279
125/430	0s 3ms/step - accuracy: 0.9926 - loss: 0.0276
147/430	0s 3ms/step - accuracy: 0.9929 - loss: 0.0272
168/430	0s 3ms/step - accuracy: 0.9934 - loss: 0.0256
190/430	0s 3ms/step - accuracy: 0.9936 - loss: 0.0254
214/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0248
235/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0246
259/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0244
281/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0243
303/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0245
326/430	0s 3ms/step - accuracy: 0.9937 - loss: 0.0244
348/430	0s 3ms/step - accuracy: 0.9936 - loss: 0.0243
373/430	0s 2ms/step - accuracy: 0.9937 - loss: 0.0238
395/430	0s 2ms/step - accuracy: 0.9937 - loss: 0.0239
417/430	0s 2ms/step - accuracy: 0.9938 - loss: 0.0234
430/430	1s 3ms/step - accuracy: 0.9939 - loss: 0.0233 - val_accuracy: 0.9754 - val_loss: 0.0907
Epoch 10/10	

1/430	9s 22ms/step	- accuracy: 1.0000	- loss: 0.0099
23/430	0s 2ms/step	- accuracy: 0.9946	- loss: 0.0266
43/430	0s 2ms/step	- accuracy: 0.9940	- loss: 0.0247
63/430	0s 3ms/step	- accuracy: 0.9927	- loss: 0.0260
78/430	0s 3ms/step	- accuracy: 0.9925	- loss: 0.0257
95/430	0s 3ms/step	- accuracy: 0.9931	- loss: 0.0237
113/430	0s 3ms/step	- accuracy: 0.9932	- loss: 0.0235
132/430	0s 3ms/step	- accuracy: 0.9933	- loss: 0.0231
152/430	0s 3ms/step	- accuracy: 0.9933	- loss: 0.0229
169/430	0s 3ms/step	- accuracy: 0.9935	- loss: 0.0220
183/430	0s 3ms/step	- accuracy: 0.9936	- loss: 0.0219
201/430	0s 3ms/step	- accuracy: 0.9937	- loss: 0.0215
219/430	0s 3ms/step	- accuracy: 0.9938	- loss: 0.0212
238/430	0s 3ms/step	- accuracy: 0.9937	- loss: 0.0212
259/430	0s 3ms/step	- accuracy: 0.9938	- loss: 0.0212
278/430	0s 3ms/step	- accuracy: 0.9938	- loss: 0.0211
300/430	0s 3ms/step	- accuracy: 0.9939	- loss: 0.0213
320/430	0s 3ms/step	- accuracy: 0.9940	- loss: 0.0212
346/430	0s 3ms/step	- accuracy: 0.9941	- loss: 0.0209
369/430	0s 3ms/step	- accuracy: 0.9942	- loss: 0.0205
392/430	0s 3ms/step	- accuracy: 0.9943	- loss: 0.0205
416/430	0s 3ms/step	- accuracy: 0.9944	- loss: 0.0203
430/430	1s 3ms/step	- accuracy: 0.9944	- loss: 0.0202 - val_accuracy: 0.9754 - val_loss: 0.0955

```

test_loss_keras, test_acc_keras = model_keras_mnist.evaluate(X_test_flat, y_test, verbose=0)
print(f"\nKeras – Exactitud en el conjunto de prueba (evidencia de ejecución): {test_acc_keras*100:.2f}%")

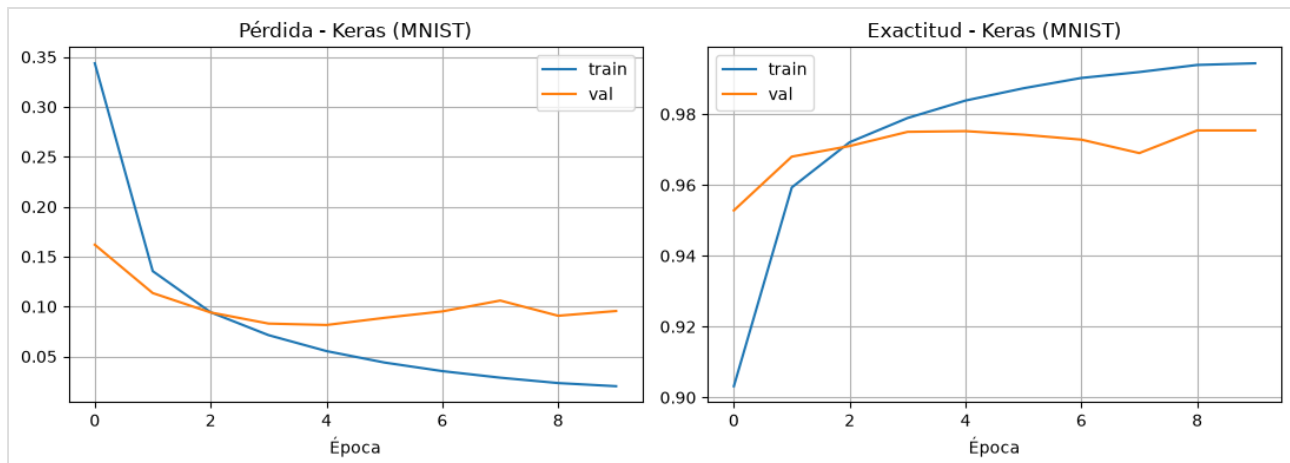
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(history_keras_mnist.history["loss"], label="train")
axes[0].plot(history_keras_mnist.history["val_loss"], label="val")
axes[0].set_title("Pérdida - Keras (MNIST)")
axes[0].set_xlabel("Época")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history_keras_mnist.history["accuracy"], label="train")
axes[1].plot(history_keras_mnist.history["val_accuracy"], label="val")
axes[1].set_title("Exactitud - Keras (MNIST)")
axes[1].set_xlabel("Época")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

```

Keras – Exactitud en el conjunto de prueba (evidencia de ejecución): 97.27%



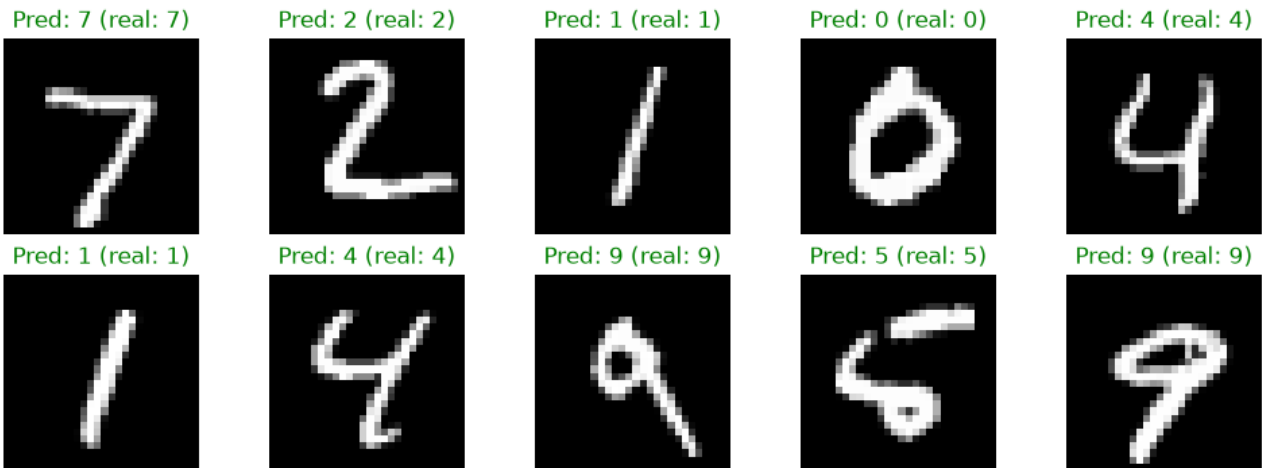
```

# Evidencia de ejecución: predicciones sobre algunas muestras de prueba
preds_keras = np.argmax(model_keras_mnist.predict(X_test_flat[:10], verbose=0), axis=1)

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_test[i], cmap="gray")
    color = "green" if preds_keras[i] == y_test[i] else "red"
    ax.set_title(f"Pred: {preds_keras[i]} (real: {y_test[i]})", color=color)
    ax.axis("off")
plt.suptitle("Predicciones del modelo Keras sobre datos de prueba")
plt.tight_layout()
plt.show()

```

Predicciones del modelo Keras sobre datos de prueba



3. Clasificación multiclase con PyTorch

Replicamos la misma arquitectura (**784 → 128 (ReLU) → 64 (ReLU) → 10**) en PyTorch. Aquí hay un detalle que vale la pena mencionar: `nn.CrossEntropyLoss` de PyTorch ya incluye internamente el cálculo de Softmax, así que la red devuelve directamente los valores "crudos" de la última capa (sin aplicar softmax explícitamente) y es la propia función de pérdida la que se encarga de esa conversión.

```
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

torch.manual_seed(42)

X_train_t = torch.tensor(X_train_flat, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
X_val_t = torch.tensor(X_val_flat, dtype=torch.float32)
y_val_t = torch.tensor(y_val, dtype=torch.long)
X_test_t = torch.tensor(X_test_flat, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.long)

train_loader = DataLoader(TensorDataset(X_train_t, y_train_t), batch_size=128, shuffle=True)

print("PyTorch:", torch.__version__)
```

PyTorch: 2.14.0+cpu

```
class TorchMNISTClassifier(nn.Module):
    """Red densa multicapa: 784 -> 128 (ReLU) -> 64 (ReLU) -> 10 (logits, sin CNN)."""

    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)  # logits crudos; CrossEntropyLoss aplica softmax internamente
        )

    def forward(self, x):
        return self.net(x)

model_torch_mnist = TorchMNISTClassifier()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model_torch_mnist.parameters(), lr=0.001)
```

```
print(model_torch_mnist)
```

```
TorchMNISTClassifier(  
  (net): Sequential(  
    (0): Linear(in_features=784, out_features=128, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=128, out_features=64, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=64, out_features=10, bias=True)  
  )  
)
```

```
def evaluate_torch(model, X, y):  
    model.eval()  
    with torch.no_grad():  
        logits = model(X)  
        preds = torch.argmax(logits, dim=1)  
        acc = (preds == y).float().mean().item()  
        loss = criterion(logits, y).item()  
    return loss, acc  
  
history_torch_mnist = {"loss": [], "val_loss": [], "accuracy": [], "val_accuracy": []}  
  
for epoch in range(10):  
    model_torch_mnist.train()  
    for xb, yb in train_loader:  
        optimizer.zero_grad()  
        logits = model_torch_mnist(xb)  
        loss = criterion(logits, yb)  
        loss.backward()  
        optimizer.step()  
  
    train_loss, train_acc = evaluate_torch(model_torch_mnist, X_train_t, y_train_t)  
    val_loss, val_acc = evaluate_torch(model_torch_mnist, X_val_t, y_val_t)  
  
    history_torch_mnist["loss"].append(train_loss)  
    history_torch_mnist["val_loss"].append(val_loss)  
    history_torch_mnist["accuracy"].append(train_acc)  
    history_torch_mnist["val_accuracy"].append(val_acc)  
  
    print(f"Época {epoch + 1:2d}/10 | loss: {train_loss:.4f} | acc: {train_acc*100:.2f}% "  
          f"| val_loss: {val_loss:.4f} | val_acc: {val_acc*100:.2f}%")
```

```
Época 1/10 | loss: 0.2235 | acc: 93.37% | val_loss: 0.2150 | val_acc: 93.64%
```

```
Época 2/10 | loss: 0.1396 | acc: 95.91% | val_loss: 0.1427 | val_acc: 95.94%
```

```
Época 3/10 | loss: 0.1185 | acc: 96.35% | val_loss: 0.1270 | val_acc: 96.26%
```

```
Época 4/10 | loss: 0.0788 | acc: 97.69% | val_loss: 0.0970 | val_acc: 97.18%
```

```
Época 5/10 | loss: 0.0745 | acc: 97.73% | val_loss: 0.1021 | val_acc: 96.80%
```

```
Época 6/10 | loss: 0.0560 | acc: 98.37% | val_loss: 0.0903 | val_acc: 97.48%
```

```
Época 7/10 | loss: 0.0395 | acc: 98.85% | val_loss: 0.0745 | val_acc: 97.70%
```

```
Época 8/10 | loss: 0.0335 | acc: 99.02% | val_loss: 0.0752 | val_acc: 97.62%
```

```
Época 9/10 | loss: 0.0256 | acc: 99.34% | val_loss: 0.0769 | val_acc: 97.76%
```

```
Época 10/10 | loss: 0.0249 | acc: 99.26% | val_loss: 0.0789 | val_acc: 97.88%
```

```
test_loss_torch, test_acc_torch = evaluate_torch(model_torch_mnist, X_test_t, y_test_t)  
print(f"\nPyTorch – Exactitud en el conjunto de prueba (evidencia de ejecución): {test_acc_torch*100:.2f}%")  
  
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
```

```

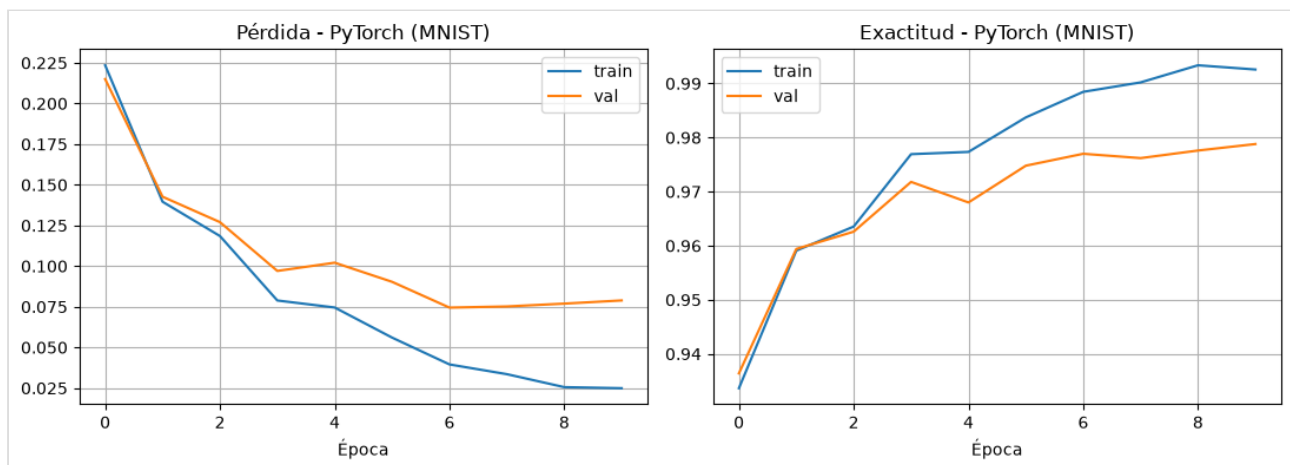
axes[0].plot(history_torch_mnist["loss"], label="train")
axes[0].plot(history_torch_mnist["val_loss"], label="val")
axes[0].set_title("Pérdida - PyTorch (MNIST)")
axes[0].set_xlabel("Época")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(history_torch_mnist["accuracy"], label="train")
axes[1].plot(history_torch_mnist["val_accuracy"], label="val")
axes[1].set_title("Exactitud - PyTorch (MNIST)")
axes[1].set_xlabel("Época")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

```

PyTorch – Exactitud en el conjunto de prueba (evidencia de ejecución): 97.68%

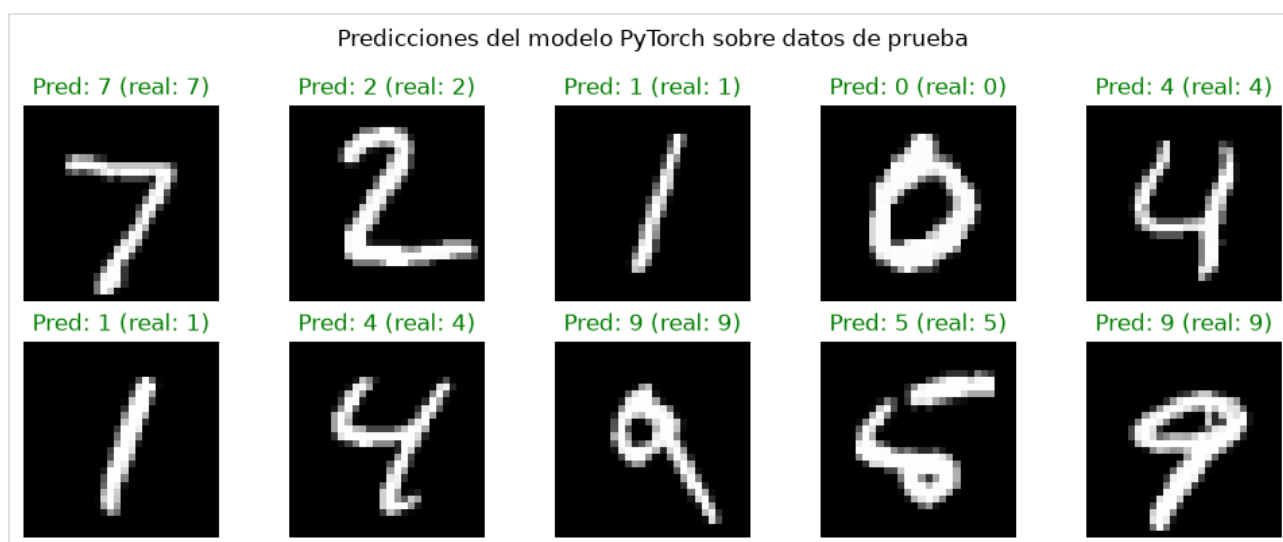


```

# Evidencia de ejecución: predicciones sobre algunas muestras de prueba
model_torch_mnist.eval()
with torch.no_grad():
    preds_torch = torch.argmax(model_torch_mnist(X_test_t[:10]), dim=1).numpy()

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_test[i], cmap="gray")
    color = "green" if preds_torch[i] == y_test[i] else "red"
    ax.set_title(f"Pred: {preds_torch[i]} (real: {y_test[i]})", color=color)
    ax.axis("off")
plt.suptitle("Predicciones del modelo PyTorch sobre datos de prueba")
plt.tight_layout()
plt.show()

```



4. Comparación de resultados y evidencias de ejecución

```
print("Resumen de exactitud sobre el conjunto de prueba de MNIST\n")
print(f"{'Framework':<20}{'Arquitectura':<30}{'Accuracy (test)':>18}")
print("-" * 68)
print(f"{'TensorFlow/Keras':<20}{'784-128-64-10 (denso)':<30}{test_acc_keras*100:>17.2f}%")
print(f"{'PyTorch':<20}{'784-128-64-10 (denso)':<30}{test_acc_torch*100:>17.2f}%")
```

Resumen de exactitud sobre el conjunto de prueba de MNIST

Framework	Arquitectura	Accuracy (test)

TensorFlow/Keras	784-128-64-10 (denso)	97.27%
PyTorch	784-128-64-10 (denso)	97.68%

5. Conclusiones

- Pasar de clasificación binaria a clasificación multiclase implica un cambio de diseño importante: la salida ya no es una sola neurona con sigmoide, sino una neurona por clase (10, en este caso) con activación Softmax, que reparte la "confianza" del modelo entre todas las opciones posibles.
- Tanto Keras como PyTorch llegan a una exactitud muy alta sobre MNIST (por encima del 97%) usando solo capas densas, sin necesidad de redes convolucionales — confirmando que una arquitectura simple es suficiente para resolver bien este problema.
- Keras resume casi todo el entrenamiento en una sola instrucción (`model.fit(...)`), mientras que en PyTorch hay que escribir el ciclo completo a mano: calcular la salida, medir el error, propagar el gradiente y actualizar los pesos. Es más código, pero también se ve con más claridad qué está pasando en cada paso.
- Un detalle técnico que vale la pena recordar: en PyTorch, `nn.CrossEntropyLoss` ya aplica Softmax por dentro, así que la red no lo hace explícitamente en su última capa — a diferencia de Keras, donde sí se declara `activation="softmax"` directamente en esa capa.
- En conjunto, este ejercicio muestra que las ideas de fondo (backpropagation, funciones de activación, descenso de gradiente) son las mismas sin importar el framework — lo que cambia es cómo se expresan en código, y ambos terminan dando resultados muy parecidos sobre el mismo problema.

4. Resumen general de resultados

Práctica	Implementación	Exactitud (accuracy, test)
Clasificación binaria (semilunas)	Perceptrón simple	86.7%
	Red de una capa — Sigmoide	88.3%
	Red de una capa — ReLU	86.7%
	MLP desde cero — Sigmoide	86.7%
	MLP desde cero — ReLU	100.0%
	TensorFlow + Keras — ReLU	100.0%
	PyTorch — ReLU	100.0%
Clasificación multiclase (MNIST)	TensorFlow + Keras (784-128-64-10)	97.27%
	PyTorch (784-128-64-10)	97.68%

5. Conclusiones generales

- El **perceptrón** y la **red de una capa** (con Sigmoide o con ReLU) solo pueden trazar fronteras rectas, así que sobre un problema que no es linealmente separable su exactitud se queda bastante por debajo (entre 85% y 88%) de lo que logra la red multicapa. Es la misma limitación que ya se conocía en teoría, pero ahora comprobada con un ejemplo más exigente que las compuertas lógicas de la entrega anterior.
- **Backpropagation** es el mismo algoritmo de fondo tanto si se programa a mano con NumPy como si lo resuelve automáticamente TensorFlow/Keras o PyTorch mediante diferenciación automática (autograd) — cambia la herramienta, no la idea matemática detrás.
- Comparando **ReLU** contra **Sigmoide** en la capa oculta del MLP, ReLU vuelve a ser la opción más efectiva porque no sufre el desvanecimiento del gradiente, y fue justo esa combinación (capa oculta + ReLU) la única que llegó al 100% de exactitud en el problema de clasificación binaria.
- Pasar a **clasificación multiclase** implica cambiar la capa de salida: en vez de una neurona con Sigmoide, se necesita una neurona por clase con activación **Softmax**, junto con una función de pérdida pensada para varias clases (entropía cruzada categórica).
- **TensorFlow/Keras** y **PyTorch** dieron resultados muy parecidos tanto en el problema binario como en MNIST, lo que confirma que la elección de framework no cambia lo que el modelo puede aprender — solo cambia el estilo de escribir el código (más declarativo en Keras, más explícito en PyTorch).
- En ningún momento se usaron redes convolucionales, tal como pedía la actividad, y aun así se logró una exactitud superior al 97% reconociendo dígitos de MNIST usando solo capas densas.